HackenProof

Expert crowdsourced security provider

Final Report

Private Security Audit

for **MPP Router by ROZO**

PREPARED BY
HackenProof

REPORT CREATED
11.09.2026

AUDIT DURATION
02.09.2026 - 08.09.2026

LANGUAGE
Rust

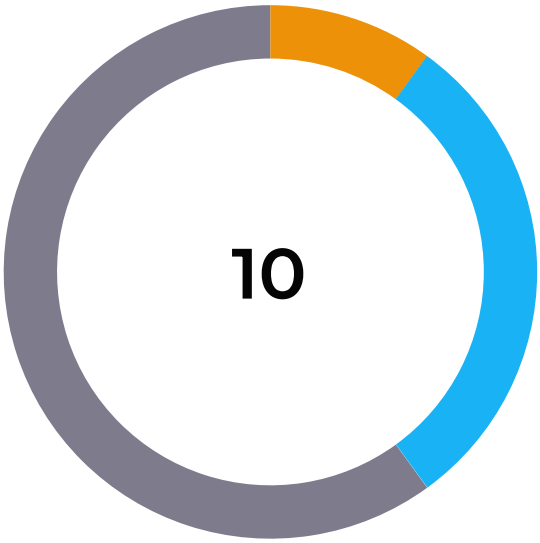
INTRODUCTION

HackenProof Audit is your solution to maximize cybersecurity through the involvement of the industry's most skilled and experienced community security researchers.

Program name	MPP Router by ROZO — Soroban Payment Channel Contracts
Type	Private Security Audit
Duration	02.09.2026 - 08.09.2026
Changelog	09.09.2026 - Preliminary Report 11.09.2026 - Final Report
Scope	https://github.com/mpprouter/one-way-channel Commit: a2eb837581544718453ca7a3ed13e768da900674 Files: contracts/account/src/lib.rs contracts/account-factory/src/lib.rs contracts/channel/src/lib.rs contracts/channel/src/event.rs contracts/channel-factory/src/lib.rs
Approved by	HackenProof team
Audited by	@tchkvsky @shealtielanz

AUDIT SUMMARY

10	10	0
Total Findings	Resolved	Informative



Critical	0
High	0
Medium	1
Low	3
None	6

FINDINGS LIST

ID	Report name	Severity	Status
ROZOSCA-7	Recipients can accept commitments that have never been proven redeemable on chain	Medium	Resolved
ROZOSCA-1	Channel implementation is not pinned to the funder's authorization	Low	Resolved
ROZOSCA-2	Refunded channels retain unpaid claims against later deposits	Low	Resolved
ROZOSCA-9	Reusing an effective closed channel bypasses the configured challenge window	Low	Resolved
ROZOSCA-3	Recipient documentation misstates commitment validation and replacement	None	Resolved
ROZOSCA-4	`deposited()` can overflow when reporting cumulative deposits	None	Resolved

ID	Report name	Severity	Status
ROZOSCA-5	Hardening Notes	None	Resolved
ROZOSCA-10	Informational 1	None	Resolved
ROZOSCA-11	Informational 2	None	Resolved
ROZOSCA-8	Opaque `personal_sign` authorizations hide the action being approved	None	Resolved

HackenProof

VULNERABILITY DETAILS

ROZOSCA-7 Recipients can accept commitments that have never been proven redeemable on chain

• MEDIUM

Target https://github.com/mpprouter/one-way-channel

Status: Resolved

Severity: • MEDIUM

Vulnerability details

[M-1] Recipients can accept commitments that have never been proven redeemable on chain

Risk Category: Payment-channel redeemability / input validation / integration safety

Context:

- contracts/channel/src/lib.rs::Contract::__constructor() lines 304-312
- contracts/channel/src/lib.rs::Commitment::verify() lines 267-272
- contracts/channel/src/lib.rs::Contract::settle() lines 480-490
- contracts/channel/src/lib.rs::Contract::close() lines 517-549
- contracts/channel/src/lib.rs recipient expectations, lines 29-37
- tools/ed25519/src/main.rs::Cmd::Verify lines 58-64

Description:

The payment-channel design requires the recipient to deliver value off chain and redeem only the latest cumulative commitment later. That is safe only if each accepted commitment is known to be redeemable by the same verifier that settle and close enforce on chain.

The constructor validates only the initial deposit amount and stores the funder-supplied commitment_key unchanged:

```
assert_with_error!(env, amount >= 0, Error::NegativeAmount);
env.storage().instance().set(&DataKey::CommitmentKey, &commitment_key);
```

The recipient checklist requires checking the waiting period, checking the amount against channel balance, monitoring Close events, and settling promptly. It does not require strict verification of the exact (amount, signature) pair or simulation of the exact settle / close call before delivering value.

This leaves two practical failure modes:

1. A funder can open a funded channel with key material that cannot produce redeemable commitments under the host verifier, then send off-chain commitments that the recipient cannot redeem.
2. A funder with an ordinary Ed25519 key can provide signatures that pass a non-strict off-chain verifier but fail Soroban's strict verifier. The shipped tools/ed25519 verify command uses the non-strict verify predicate, so it can report valid for a commitment that settle and close reject.

The second case is why a constructor-only key check is not sufficient. A key may be valid while a particular signature is still unredeemable on chain.

Impact:

A recipient can deliver off-chain goods or services against commitments that later cannot settle or close. The funder does not steal recipient funds already on chain, but can receive off-chain value and then recover the remaining escrow after the refund waiting period. The impact is bounded by the value the recipient delivers before discovering that the commitments are not redeemable.

Proof of Concept:

Simplified scenario:

1. The funder opens a channel with a funded token balance.
2. The recipient follows the documented checks and accepts commitments without a host-equivalent verification step.
3. The recipient delivers off-chain value.
4. settle / close rejects the commitment, and the funder later calls close_start and refund.

Retained validation covers both the invalid-key and ordinary-key mismatch cases:

- test_M1a_SmallOrderKeyRejectedByChannel
- test_M1b_NormalKeyVerifierDivergence
- test_M1c_NormalKeyStrictnessMismatchAgainstChannelWasm

Recommendation:

Make commitment redeemability an explicit protocol requirement:

- Update recipient-facing documentation and SDK examples to require strict, host-equivalent verification of every exact commitment before delivery, or preflight simulation of the exact settle / close call.
- Change the shipped verifier to use strict verification:

```
- Cmd::Verify { pkey, msg, sig } => match pkey.verify(&msg.0, &sig) {
+ Cmd::Verify { pkey, msg, sig } => match pkey.verify_strict(&msg.0, &sig) {
```

- Consider adding a constructor proof-of-possession check over a fixed channel-creation message. This catches invalid, small-order, or unowned key material at channel creation, but it does not replace per-commitment strict verification.

Validation steps

M-1. Recipients can accept commitments that have never been proven redeemable on chain

Commands:

```
cargo test --manifest-path audit-tests/report-findings/Cargo.toml
test_M1a_SmallOrderKeyRejectedByChannel -- --nocapture
cargo test --manifest-path audit-tests/report-findings/Cargo.toml
test_M1b_NormalKeyVerifierDivergence -- --nocapture
cargo test --manifest-path audit-tests/report-findings/Cargo.toml
test_M1c_NormalKeyStrictnessMismatchAgainstChannelWasm -- --nocapture
```

Expected result:

- test_M1a_SmallOrderKeyRejectedByChannel shows a small-order commitment key can look valid to a non-strict verifier while settle and close reject it on chain; the funder later refunds the full escrow.

- test_M1b_NormalKeyVerifierDivergence uses an ordinary, non-weak Ed25519 key and constructs signatures that pass non-strict verification but fail verify_strict for multiple messages.
- test_M1c_NormalKeyStrictnessMismatchAgainstChannelWasm shows the channel WASM rejects the forged strict-invalid signature while accepting genuine signatures under the same normal key before and after the failed attempt.

Primary test function:

```
#[test]
#[allow(non_snake_case)]
fn test_M1c_NormalKeyStrictnessMismatchAgainstChannelWasm() {
    use curve25519_dalek::scalar::Scalar;
    use ed25519_dalek::Verifier;
    use sha2::Sha512;

    let env = Env::default();
    env.mock_all_auths_allowing_non_root_auth();
    let key = EdSigningKey::from_bytes(&[42u8; 32]);
    let vk = key.verifying_key();
    assert!(!vk.is_weak());
    let (token_addr, token, asset_admin) = create_token(&env);
    let funder = Address::generate(&env);
    let recipient = Address::generate(&env);
    asset_admin.mint(&funder, &1_000);
    let channel = env.register(channel_contract::WASM, (&token_addr, &funder,
&BytesN::from_array(&env, &vk.to_bytes()), &recipient, 1_000i128, 100u32));
    let client = channel_contract::Client::new(&env, &channel);

    client.settle(&100, &commitment_sig(&env, &key, &channel, 100));
    assert_eq!(token.balance(&recipient), 100);

    let payload = client.prepare_commitment(&300);
    let mut message = std::vec![0u8; payload.len() as usize];
    payload.copy_into_slice(&mut message);
    let expanded = Sha512::digest(key.to_bytes());
    let mut scalar_bytes = [0u8; 32];
    scalar_bytes.copy_from_slice(&expanded[..32]);
    scalar_bytes[0] &= 248;
    scalar_bytes[31] &= 127;
    scalar_bytes[31] |= 64;
    let secret_scalar = Scalar::from_bytes_mod_order(scalar_bytes);
    let mut signature = [0u8; 64];
    signature[0] = 1;
    let mut challenge = Sha512::new();
    challenge.update(&signature[..32]);
    challenge.update(vk.to_bytes());
    challenge.update(&message);
    let k = Scalar::from_bytes_mod_order_wide(&challenge.finalize().into());
    signature[32..].copy_from_slice(&(k * secret_scalar).to_bytes());
    let dalek_signature = ed25519_dalek::Signature::from_bytes(&signature);
    assert!(vk.verify(&message, &dalek_signature).is_ok());
    assert!(vk.verify_strict(&message, &dalek_signature).is_err());
```

```
    let signature = BytesN::from_array(&env, &signature);
    assert!(client.try_settle(&300, &signature).is_err());
    assert!(client.try_close(&300, &signature).is_err());
    assert_eq!(client.withdrawn(), 100);
    assert_eq!(client.balance(), 900);
    assert_eq!(token.balance(&recipient), 100);

    // A valid signature still settles, ruling out bad configuration or auth.
    client.settle(&200, &commitment_sig(&env, &key, &channel, 200));
    assert_eq!(client.withdrawn(), 200);
    client.close_start();
    env.ledger().with_mut(|l| l.sequence_number += 100);
    client.refund();
    assert_eq!(token.balance(&funder), 800);
    assert_eq!(token.balance(&recipient), 200);
}
```

Manual checks:

```
rg -n "ed25519_verify|CommitmentKey|Recipient" contracts/channel/src/lib.rs
rg -n "pkey.verify" tools/ed25519/src/main.rs
```

Expected result:

- The channel stores commitment_key without a proof-of-possession or per-signature preflight requirement.
- tools/ed25519 verify calls non-strict pkey.verify, not verify_strict.

HackenProof

VULNERABILITY DETAILS

ROZOSCA-1 Channel implementation is not pinned to the funder's authorization

• LOW

Target https://github.com/mpprouter/one-way-channel

Status: Resolved

Severity: • LOW

Vulnerability details

Affected contract: Channel factory

Functions: open, set_wasm

Summary: An unconsumed, unexpired funder authorization can remain usable after the administrator changes the channel implementation. The authorized open arguments do not include the expected WASM hash.

Description: In contracts/channel-factory/src/lib.rs, lines 73–98, set_wasm lets the authorized administrator change the stored hash. open authenticates the funder and reads that mutable hash when deploying:

```
from.require_auth();
let wasm_hash = Self::wasm_hash(env);
// ...
.deploy_v2(wasm_hash, (token, from, commitment_key, to, amount,
refund_waiting_period));
```

If the replacement preserves the required constructor and token-transfer authorization tree, a relayer can build a transaction for it using a funder authorization signed while the factory selected the previous implementation.

Recommendation:

Add expected_wasm_hash to the authenticated open arguments and reject a mismatch before deployment. Keep the existing funder authorization and complete constructor arguments intact. Immutable per-version factories are an alternative.

Validation steps

Not needed.

HackenProof

VULNERABILITY DETAILS

ROZOSCA-2 Refunded channels retain unpaid claims against later deposits

• LOW

Target https://github.com/mpprouter/one-way-channel

Status: Resolved

Severity: • LOW

Vulnerability details

Affected contract: Channel

Functions: refund, settle, close, top_up, withdraw

Summary: Refund drains the channel's current balance but does not retire unpaid commitments. An old commitment can therefore claim tokens later deposited into the same channel address.

Description: refund, lines 586–615, leaves WithdrawnAmount unchanged. settle remains callable, and withdraw pays the unpaid portion of a signed cumulative amount from the available balance.

The documentation says recovery of accidental post-close deposits without explaining this competing claim. However, continued operations after closure are explicitly supported, and an original test intentionally tops up after an automatic refund.

Impact: A recipient can race the funder's attempt to recover a later accidental deposit. The sender may lose that new deposit despite expecting the refunded address to be inactive.

Recommendation:

Choose an explicit policy and align the implementation, documentation, and tests:

- Terminal refund: Record a finalized state after the intended final refund and reject later settlement and top-ups. Preserve a recovery path for direct token transfers, which cannot generally be prevented by disabling top_up.
- Persistent entitlement: Clearly state that refund removes liquidity but does not cancel unpaid commitments, and that reusing the address exposes new deposits to those commitments.

Do not set WithdrawnAmount = i128::MAX to represent finalization. It falsifies payment accounting and causes deposited() to overflow after any positive refill.

Validation steps

Not needed.

HackenProof

VULNERABILITY DETAILS

ROZOSCA-9 Reusing an effective closed channel bypasses the configured challenge window

• LOW

Target https://github.com/mpprouter/one-way-channel

Status: Resolved

Severity: • LOW

Vulnerability details

[L-2] Reusing an effective closed channel bypasses the configured challenge window

Risk Category: Channel lifecycle / business logic

Context:

- contracts/channel/src/lib.rs::Contract::top_up() lines 337-346
- contracts/channel/src/lib.rs::Contract::settle() lines 456-490
- contracts/channel/src/lib.rs::Contract::close() lines 517-549
- contracts/channel/src/lib.rs::Contract::close_start() lines 564-583
- contracts/channel/src/lib.rs::Contract::refund() lines 597-615

Description:

The contract allows a closed channel to receive new balance and allows the recipient to settle while balance remains. However, once CloseEffectiveAtLedger is already effective, the funder cannot arm a fresh waiting period
because close_start returns AlreadyClosed.

This creates a reuse hazard. If new business is routed through a channel whose prior close is already effective, the funder can refund the new balance without giving the recipient the configured reaction window.

This is distinct from a pending close being resolved by close. A pending close can still be handled by the recipient. The issue is reuse after the effective close point has already passed.

Impact:

The recipient can lose the off-chain value associated with new business accepted on a previously closed channel. The impact is Low because the effective close is public state and careful clients can reject reused channels, but the current API and documentation do not make reuse terminal.

Proof of Concept:

Simplified scenario:

1. A channel reaches an effective closed state.
2. The funder or an integration routes fresh balance / fresh business through the same channel.
3. The recipient accepts a new off-chain commitment expecting the configured waiting period.
4. The funder immediately calls refund because the prior close is already effective.

Retained validation:

- test_L2a_ReusedEffectiveClosedChannelBypassesChallengeWindow
- test_L2b_NewBalanceCanArriveIntoAlreadyRefundableChannel
- test_L2c_ReusedChannelCannotBeRearmedForFreshWindow

Recommendation:

Choose one lifecycle model and enforce it in code:

- If closed channels are terminal, reject top_up, settle, and close after CloseEffectiveAtLedger is effective.
- If reuse is intended, add an explicit channel session / epoch and include it in the commitment payload. Reuse should clear or replace the prior close state and make old commitments invalid for the new session.

Validation steps

L-2. Reusing an effective closed channel bypasses the configured challenge window

Commands:

```
cargo test --manifest-path audit-tests/report-findings/Cargo.toml
test_L2a_ReusedEffectiveClosedChannelBypassesChallengeWindow -- --nocapture
cargo test --manifest-path audit-tests/report-findings/Cargo.toml
test_L2b_NewBalanceCanArriveIntoAlreadyRefundableChannel -- --nocapture
cargo test --manifest-path audit-tests/report-findings/Cargo.toml
test_L2c_ReusedChannelCannotBeRearmedForFreshWindow -- --nocapture
```

Expected result:

- test_L2a_ReusedEffectiveClosedChannelBypassesChallengeWindow passes and demonstrates the deterministic reuse sequence.
- The two additional L-2 tests pass and print the reused-channel evidence: fresh balance can arrive after a prior close is already effective, and the channel cannot be re-armed for a fresh challenge window.

Primary test function:

```
#[test]
#[allow(non_snake_case)]
fn test_L2a_ReusedEffectiveClosedChannelBypassesChallengeWindow() {
    let env = Env::default();
    env.mock_all_auths_allowing_non_root_auth();

    let commitment_key = EdSigningKey::from_bytes(&[4u8; 32]);
    let commitment_pub = BytesN::from_array(&env,
&commitment_key.verifying_key().to_bytes());

    let (token_addr, token, asset_admin) = create_token(&env);
    let funder = Address::generate(&env);
    let to = Address::generate(&env);
    asset_admin.mint(&funder, &1_500);

    // A deliberately generous six-day challenge window.
    let channel = env.register(channel_contract::WASM, (&token_addr, &funder,
&commitment_pub, &to, 1_000i128, 100_000u32));
    let client = channel_contract::Client::new(&env, &channel);
```

```
    // Recipient draws down and closes - the normal, intended flow.
    let sig = commitment_sig(&env, &commitment_key, &channel, 300i128);
    client.close(&300i128, &sig);
    assert_eq!(token.balance(&to), 300);

    // Funder re-funds the channel to keep using it (the flow test_close_twice covers).
    client.top_up(&500i128);

    // Every exported getter still describes a healthy, protected channel.
    assert_eq!(client.refund_waiting_period(), 100_000);
    assert_eq!(client.balance(), 500);
    assert_eq!(client.withdrawn(), 300);
    std::println!("getters report: refund_waiting_period={}, balance={}",
client.refund_waiting_period(), client.balance());

    // The funder issues a fresh cumulative commitment for the refilled session.
    // It is a normal strict-valid signature; the recipient can verify it and
    // deliver another 500 of off-chain value.
    let fresh_sig = commitment_sig(&env, &commitment_key, &channel, 800i128);

    // But the protection is gone and cannot be re-armed.
    assert!(client.try_close_start().is_err(), "close_start can never re-arm");

    // Funder refunds in the very next ledger, with no Close event and no notice.
    env.ledger().with_mut(|l| l.sequence_number += 1);
    client.refund();
    assert_eq!(token.balance(&to), 300);
    assert_eq!(client.balance(), 0);

    // The fresh commitment still verifies on chain, but settlement is now a
    // successful no-op because the funder won the zero-window refund race.
    client.settle(&800i128, &fresh_sig);
    assert_eq!(client.withdrawn(), 300);
    assert_eq!(token.balance(&to), 300);
    std::println!("funder refunded the new deposit with a zero-ledger window");
}
```

Manual checks:

```
rg -n "top_up|close_start|refund|close_effective_at_ledger" contracts/channel/src/
lib.rs
```

Expected result:

- top_up, settle, and close remain callable in closed states.
- close_start returns AlreadyClosed once the prior close is effective.
- refund can transfer the live balance immediately after the effective ledger.

I-1. Long-lived factories and idle accounts depend on manual TTL maintenance

Validation type: inspection-led. No standalone audit-package test is attached; the production test_extend case is used only as a sanity check for existing account TTL behavior.

Commands:

```
cargo test -p account test_extend
```

Expected result:

- Account extend passes in the production suite; direct code review confirms account and channel extend entry points are permissionless and work as intended.
- This does not cover the factories, which have no equivalent extension entry point.

Manual checks:

```
rg -n "extend_ttl|pub fn extend|__constructor|wasm_hash" contracts/account/src/lib.rs contracts/channel/src/lib.rs contracts/account-factory/src/lib.rs contracts/channel-factory/src/lib.rs
```

Expected result:

- Account and channel instances extend TTL on active paths.
- Account factory and channel factory store instance data but do not expose a TTL maintenance function.

HackenProof

VULNERABILITY DETAILS

ROZOSCA-3 Recipient documentation misstates commitment validation and replacement

• NONE

Target https://github.com/mpprouter/one-way-channel

Status: Resolved

Severity: • NONE

Vulnerability details

Affected files: contracts/channel/src/lib.rs and README.md

Summary: The recipient documentation contains an incorrect funding check and ambiguous commitment-replacement wording. These discrepancies can cause integrations to reject valid payments and users to misunderstand whether an existing payment authorization can be reduced.

Description: The recipient guidance at lines 31–34 instructs recipients to verify that a commitment amount is less than the channel balance. However, commitments represent cumulative payments, while the channel balance excludes amounts already withdrawn. The relevant funding requirement is the unpaid portion of the commitment.

For example, a channel funded with 500 has a remaining balance of 300 after the recipient withdraws 200. A new cumulative commitment for 400 requires only an additional 200 and is fully funded. Comparing 400 directly against the remaining balance of 300 incorrectly rejects this payment. The strict “less than” wording also excludes a commitment whose unpaid amount exactly equals the available balance.

The payment description at lines 104–112 states that each new commitment replaces the previous one. The implementation does not revoke earlier signatures. If a funder signs a commitment for 500 and subsequently signs one for 100, the recipient can still submit the 500 commitment and collect its unpaid portion, subject to the available balance. Withdrawal accounting prevents repeated collection of amounts already paid.

Both statements are also present in README.md.

Impact: Clients implementing the documented balance check can reject fully funded payments, interrupting payment flows and unnecessarily restricting channel use.

Funders may incorrectly assume that issuing a lower commitment reduces an earlier authorization. The recipient retains the ability to collect the previously authorized unpaid amount when channel funds are available.

Recommendation:

Update the Rust documentation and README to compare the unpaid portion of a nonnegative commitment against the available balance:

```
max(commitment_amount - withdrawn, 0) <= channel_balance
```

Clarify that commitment amounts are cumulative, previously signed commitments remain valid, and issuing a lower amount does not revoke an earlier higher authorization. Instruct signing clients to maintain nondecreasing cumulative amounts.

Retain the separate requirements to validate channel identity, configuration, and closure status before accepting payments.

Validation steps

Not needed.

HackenProof

VULNERABILITY DETAILS

ROZOSCA-4 `deposited()` can overflow when reporting cumulative deposits

• NONE

Target https://github.com/mpprouter/one-way-channel

Status: Resolved

Severity: • NONE

Vulnerability details

Affected files: contracts/channel/src/lib.rs

Affected function: deposited

Summary: The deposited() getter adds the current channel balance to cumulative recipient withdrawals using i128 arithmetic. The sum can exceed i128::MAX even when both individual values are valid, causing the getter to abort.

Description: At lines 422–424, the contract calculates the reported total as follows:

```
pub fn deposited(env: &Env) -> i128 {
    Self::balance(env) + Self::withdrawn(env)
}
```

The live balance and cumulative withdrawals are individually represented as i128, but their sum is not constrained to that range. The release profile enables overflow checks, so an out-of-range addition aborts execution.

For example, a recipient can withdraw 500, leaving WithdrawnAmount = 500, after which the channel receives a top-up bringing its balance to i128::MAX. Both values remain individually representable, but deposited() attempts to calculate i128::MAX + 500 and fails. This sequence was reproduced using a Stellar Asset Contract.

Closing the channel with the already-paid 500 commitment successfully refunded the live balance. After the refund, deposited() returned 500 again.

Impact: Applications or contracts relying on deposited() can fail to retrieve the total or complete operations that require this getter when the sum exceeds its supported range.

The condition requires extreme amounts for ordinary token denominations. It does not corrupt the recorded withdrawals, and settlement and refund operations do not depend on this getter. The getter failure therefore does not itself prevent fund recovery.

Recommendation:

Define the supported range and error behavior of the reported cumulative total.

If the full total must remain readable across all valid combinations of balance and withdrawals, use a wider representation such as I256 and update the contract interface and consumers accordingly.

If the return type must remain i128, use checked addition with an explicit range error and document the limitation. Checked addition provides controlled error handling but does not make an oversized result representable. Avoid silent wrapping or saturation, which would return an inaccurate total.

Validation steps

Not needed.

HackenProof

VULNERABILITY DETAILS

ROZOSCA-5 Hardening Notes

• NONE

Target https://github.com/mpprouter/one-way-channel

Status: Resolved

Severity: • NONE

Vulnerability details

One-Way Channel — Hardening and Integration Recommendations

Scope: Account, account factory, channel, channel events, and channel factory; related documentation and integration guidance

This report presents pre-deployment recommendations for payment documentation, public contract interfaces, factory maintenance, event indexing, numeric range handling, and integration behavior. Each entry describes the affected behavior, its impact, and the proposed improvement. The entries are ordered by implementation priority; the classifications distinguish informational observations from optional integration guidance.

ID	Recommendation	Classification
H-01	Correct cumulative-payment documentation	Informational — documentation
H-02	Expose the close deadline and commitment key through public getters	Informational — interface
H-03	Maintain factory TTL and support restoration	Informational — operations
H-04	Add party-address topics for event discovery and monitoring	Informational — event indexing
H-05	Define the supported range of `deposited()`	Informational — correctness
H-06	Define safe refund waiting-period acceptance rules	Informational — configuration
H-07	Document repeated closure requests and choose deadline behavior	Informational — optional behavior change
H-08	Validate automatic-refund return decoding explicitly	Informational — optional ABI hardening
H-09	Make account authorization understandable in signing clients	Integration guidance — clients outside the reviewed scope

H-01 — Correct cumulative-payment documentation

Affected components:

Channel Rust documentation, README.md, signing and recipient integrations

Classification: Informational — documentation

Summary: Correct the recipient funding check and clarify that issuing a new commitment does not invalidate previously signed commitments.

Description: The recipient guidance compares the entire cumulative commitment amount with the remaining channel balance. After a partial withdrawal, only the unpaid portion requires funding.

For example, a channel funded with 500 has a balance of 300 after the recipient withdraws 200. A cumulative commitment for 400 requires another 200 and is fully funded. Comparing 400 with the live balance of 300 incorrectly rejects the payment. The strict less-than wording also rejects a payment whose unpaid amount exactly equals the available balance.

The payment description states that each new commitment replaces the previous one. Earlier signatures remain valid: signing a commitment for 100 after signing one for 500 does not revoke the unpaid entitlement under the 500 commitment. Withdrawal accounting prevents repeated collection of amounts already paid. Both statements also appear in README.md.

Impact: Integrations implementing the documented funding check can reject valid payments and interrupt channel use. Funders may also incorrectly assume that signing a lower cumulative amount reduces an earlier authorization.

Description: Update the Rust documentation and README to compare the unpaid portion of a nonnegative commitment with the current balance:

```
max(commitment_amount - withdrawn, 0) <= channel_balance
```

Explain that commitments are cumulative and issuing a lower amount does not revoke an earlier higher signature. Instruct signing clients to maintain nondecreasing cumulative amounts.

Retain the requirements to validate channel identity, configuration, and closure status, and to settle promptly when closure begins. A balance check is a point-in-time observation, not a reservation of funds.

H-02 — Expose the close deadline and commitment key through public getters

Affected component: Channel contract interface

Classification: Informational — interface

Summary: Expose the stored close deadline and commitment key through read-only contract methods so integrations can inspect them through the public interface.

Description: The close_effective_at_ledger helper is defined in a plain implementation block and is not exported through #[contractimpl]. The public getter set also lacks a method returning the configured commitment key.

The key is included in the opening event, and closure events report the effective deadline. Off-chain integrations can inspect current instance storage using Stellar RPC's state-query interface, but this requires decoding the storage layout. Another contract cannot obtain these values through the channel's current public read-only methods.

Impact: Integrations must depend on event history or direct storage decoding for values relevant to channel validation and refund timing. Contracts implementing deadline-dependent logic lack a direct query method. This is an interface limitation; settlement remains callable without first querying the deadline.

Recommendation:

Export close_effective_at_ledger() -> Option<u32> and commitment_key() -> BytesN<32> through the contract interface. Preserve None for a channel without an initiated closure and return the stored deadline for pending and effective closure.

Document the methods and verify that they report the constructor key and each supported closure transition accurately. Use the latest stored deadline when reconciling state: recipient-initiated close can make refund eligibility immediate, so selecting the maximum historical deadline is incorrect. Read-only access should complement timely event monitoring and settlement.

H-03 — Maintain factory TTL and support restoration

Affected components:

Account factory and channel factory

Classification: Informational — operations

Summary: Add factory TTL maintenance and a tested restoration workflow to reduce interruptions when factory state is archived.

Description: Neither the account factory nor the channel factory calls extend_ttl. Ordinary activity alone does not extend their instance lifetime. Once the instance is archived, use requires restoration of the relevant entries, whether through supported automatic restoration or an explicit restoration transaction.

Instance storage is restorable; archival does not permanently erase the stored configuration. The account and channel contracts already contain instance TTL-extension helpers, which extend their instance and associated code entries under the reviewed SDK behavior.

Impact: Factory calls may require additional restoration resources or fail when the submitting client does not handle restoration. Newly deployed accounts or channels can be delayed until that requirement is satisfied. Existing child contracts operate independently of their factories, subject to their own lifetime requirements.

Recommendation:

Extend the factory instance TTL during appropriate operations and consider an explicit maintenance method for periods of low activity. Include factory instances, their code, and the child WASM entries needed for deployment in operational lifetime monitoring.

Document the restoration workflow supported by the deployment network and client stack. Test archival and restoration, confirm configuration is preserved, and verify that account and channel creation resume afterward.

H-04 — Add party-address topics for event discovery and monitoring

Affected component:

Channel event definitions in contracts/channel/src/event.rs

Classification: Informational — event indexing

Summary: Index funder and recipient addresses as event topics to support efficient discovery and monitoring of channels associated with a party.

Description: The event definitions place party addresses in event data rather than fields annotated with #[topic]. Event-name topics support filtering by event type, but the current schemas do not support topic filters for a particular funder or recipient.

Stellar RPC event responses already identify the emitting contract and ledger and provide ordering information. A monitor with known channel addresses can filter by contract. Discovering channels for a recipient through opening events currently requires fetching a broader event set and inspecting its data.

Impact: Recipients and indexers need more event processing and channel-address tracking to discover and monitor their channels. Discovery remains possible, but the event layout makes party-specific filtering less efficient. The observation concerns integration cost and observability, rather than an inability to identify the channel that emitted an event.

Recommendation:

Promote Open.from, Open.to, Deposit.from, Withdraw.to, and Refund.from to topics. Consider including the recipient as a topic on Close when monitoring closures across multiple channels is required.

Document that the Close event is emitted by both close_start and recipient-initiated

Validation steps

Not needed.

HackenProof

VULNERABILITY DETAILS

ROZOSCA-10 Informational 1

• NONE

Target https://github.com/mpprouter/one-way-channel

Status: Resolved

Severity: • NONE

Vulnerability details

[I-1] Long-lived factories and idle accounts depend on manual TTL maintenance

Risk Category: Soroban storage lifecycle / operational reliability

Context:

- contracts/account/src/lib.rs TTL constants, lines 56-61
- contracts/account/src/lib.rs::Contract::extend() lines 110-120
- contracts/account/src/lib.rs::Contract::__check_auth() lines 135-136 and 172-175
- contracts/channel/src/lib.rs TTL constants, lines 215-220
- contracts/channel/src/lib.rs::Contract::extend() lines 349-360
- contracts/account-factory/src/lib.rs::FactoryContract::__constructor() / wasm_hash() lines 56-68
- contracts/channel-factory/src/lib.rs::FactoryContract::__constructor() lines 42-45

Description: Channel and account instances provide explicit extend functions and extend TTL on active paths. The two factories do not extend instance storage as a side effect of normal calls, and an account deployed ahead of first use can remain idle until it needs restoration.

This is not a direct theft path. The practical risk is operational: integrations that expect deterministic account addresses or factory availability over long periods must actively maintain instance storage, otherwise calls can fail or require restoration during onboarding or recovery. A third party can deploy an account for a user, but cannot bind it to a different EVM signer.

Recommendation:

Add factory extend functions and extend factory TTL on read/write entry points. Document that clients or keepers must maintain TTL for factories, long-lived channels, and accounts deployed before first use.

[I-2] Channel and factory documentation omits important runtime behavior

Risk Category: Documentation / integration safety

Context:

- contracts/channel/src/lib.rs state diagram and lifecycle note, lines 39-52
- contracts/channel/src/lib.rs::Contract::settle() docs, lines 456-479
- contracts/channel/src/lib.rs::Contract::refund() docs, lines 586-595
- contracts/channel-factory/src/lib.rs::FactoryContract::open() lines 81-99

Description: Several runtime behaviors matter for safe integrations but are not stated with enough precision in the public comments:

- A closed channel can still receive tokens and can still process settle / close until refund drains the balance.
- Direct token transfers to the channel do not emit channel Deposit events.
- refund_waiting_period = 0 allows near-immediate refund.
- The channel factory admin can change the implementation used for future channels.
- A one-call wrapper can deploy an account and perform its first authorized action in the same Soroban operation if it constructs the call tree correctly.

These points are not independent vulnerabilities, but they affect how clients should monitor, quote, and route channel activity. Incorrect assumptions can produce avoidable failed calls, missed monitoring, or unsafe channel reuse even though the underlying code behavior is deterministic and externally observable.

Recommendation:

Update the module-level documentation and integration guide to state the runtime behavior explicitly. Keep the documented state diagram aligned with actual callable paths, especially closed-channel reuse, direct transfers, factory implementation mutability, and zero-length waiting periods.

[I-3] Some no-payment paths return success without an event

Risk Category: Observability / accounting integration

Context:

- contracts/channel/src/lib.rs::Contract::close() lines 541-548
- contracts/channel/src/lib.rs::Contract::withdraw() lines 632-645

Description: Two successful call paths can make no token movement and emit no channel event:

- close catches an auto-refund failure through try_transfer(...).is_ok() and continues.
- settle / close with no positive payout returns successfully without a Withdraw event.

The no-op behavior is sometimes intentional, especially for older commitments or empty balances. The issue is observability: indexers that treat success as a payment/refund signal cannot distinguish a no-op from a successful value transfer without reading token events or state. Clients can misreport payment or refund completion, but contract funds remain safe because token movement is governed by the token contract and channel accounting.

Recommendation:

Emit explicit no-op events or return structured outcomes for no-payment paths. At minimum, document that callers must inspect token events or state deltas rather than relying only on channel-call success.

[I-4] deposited() can trap on overflow under the release profile

Risk Category: Arithmetic / getter reliability

Context:

- contracts/channel/src/lib.rs::Contract::deposited() lines 420-424

Description: deposited() returns balance + withdrawn. The rest of the value-moving code caps payouts and updates WithdrawnAmount by the amount actually transferred. This getter is the only unbounded arithmetic site identified in the channel module.

Under a release profile with overflow checks enabled, an extreme token balance plus a large withdrawn amount can trap the getter. This affects read availability for an extreme state, but it does not create unauthorized withdrawal, double spend, or accounting corruption in the settlement path.

Recommendation:

Use checked or saturating arithmetic and return a typed error if the sum cannot be represented:

```
Self::balance(env)
  .checked_add(Self::withdrawn(env))
  .ok_or(Error::AmountOverflow)?
```

[I-5] Channel factory implementation updates are unannounced and unbound to funder authorization

Risk Category: Factory governance / deployment integrity

Context:

- contracts/channel-factory/src/lib.rs::FactoryContract::__constructor() lines 42-45
- contracts/channel-factory/src/lib.rs::FactoryContract::set_wasm() lines 67-79
- contracts/channel-factory/src/lib.rs::FactoryContract::open() lines 87-99

Description: The channel factory admin can change the wasm hash used for future channel deployments. This is a documented trust assumption, not an access-control bypass. The hardening gap is that set_wasm emits no event, and open does not bind the funder's authorization to the expected wasm hash.

If a client caches an expected implementation hash but does not read the factory immediately before signing, the factory may deploy a different channel implementation than the one the funder expected. Future channel deployments therefore depend on the factory admin and client-side hash checks. Existing deployed channels are not upgraded or changed by set_wasm, and no unauthorized admin path was found.

Recommendation:

Emit an event on every wasm-hash update and allow open callers to pass an expected_wasm_hash that must equal the stored hash:

```
assert_with_error!(env, expected_wasm_hash == Self::wasm_hash(env),
Error::UnexpectedWasmHash);
```

Validation steps

I-1. Long-lived factories and idle accounts depend on manual TTL maintenance

Validation type: inspection-led; no standalone audit-package test.

Commands:

```
cargo test -p account test_extend
```

rg -n "extend_ttl|pub fn extend|__constructor|wasm_hash" contracts/account/src/lib.rs contracts/channel/src/lib.rs contracts/account-factory/src/lib.rs contracts/channel-factory/src/lib.rs

Expected result: account/channel instances expose TTL extension paths; both factories store instance data but expose no TTL maintenance function.

I-2. Channel and factory documentation omits important runtime behavior

Validation type: inspection-only; no standalone audit-package test.

Commands:

```bash
rg -n "State diagram|top_up|settle|close_start|refund|set_wasm|open_account|refund_waiting_period" contracts/account/src/lib.rs contracts/account-factory/src/lib.rs contracts/channel/src/lib.rs contracts/channel-factory/src/lib.rs
```

Expected result: comments describe the lifecycle, while source review confirms extra integration-relevant behavior: direct transfers emit no channel Deposit, closed channels can still receive balance, set_wasm changes future channel code, and zero waiting periods are accepted.

I-3. Some no-payment paths return success without an event

Commands:

```
cargo test -p channel test_settle_older_commitment_no_op
cargo test -p channel test_close_full_balance_no_refund_event
cargo test --manifest-path audit-tests/report-findings/Cargo.toml test_I3_SuccessfulCloseCanEmitNoEvents -- --nocapture
```

Expected result: older commitments and full-balance closes can succeed as no-payment paths; the report-finding test shows a successful close with no channel value event.

Primary test function:

```
#[test]
#[allow(non_snake_case)]
fn test_I3_SuccessfulCloseCanEmitNoEvents() {
    let env = Env::default();
    env.mock_all_auths_allowing_non_root_auth();

    let key = EdSigningKey::from_bytes(&[4u8; 32]);
    let commitment_pub = BytesN::from_array(&env, &key.verifying_key().to_bytes());
    let (token_addr, token, asset_admin) = create_token(&env);
    let funder = Address::generate(&env);
    let to = Address::generate(&env);
    asset_admin.mint(&funder, &1_000);

    let channel = env.register(channel_contract::WASM, (&token_addr, &funder, &commitment_pub, &to, 1_000i128, 100u32));
    let client = channel_contract::Client::new(&env, &channel);
    let sig = commitment_sig(&env, &key, &channel, 600i128);

    client.close(&600i128, &sig);
    let first = channel_event_names(&env, &channel);
    std::println!("first close emitted {}: {:?}", first.len(), first);
```

```
    client.close(&600i128, &sig);
    let second = channel_event_names(&env, &channel);
    std::println!("second close emitted {}: {:?}", second.len(), second);

    assert_eq!(first.len(), 3, "first close emits withdraw + close + refund");
    assert!(second.is_empty(), "the second successful close emitted nothing");
    assert_eq!(token.balance(&channel), 0);
}
```

Manual check: rg -n "try_transfer|payout > 0|publish_event" contracts/channel/src/lib.rs.

I-4. deposited() can trap on overflow under the release profile

Commands:

```
cargo test --manifest-path audit-tests/report-findings/Cargo.toml
test_I4a_DepositedOverflowWithStellarAssetContract -- --nocapture
cargo test --manifest-path audit-tests/report-findings/Cargo.toml
test_I4b_DepositedTrapsOnUnboundedAddition -- --nocapture
```

Expected result: the getter traps when balance + withdrawn exceeds i128::MAX; refund and lower-balance recovery still work.

Primary test function:

```
#[test]
#[allow(non_snake_case)]
fn test_I4a_DepositedOverflowWithStellarAssetContract() {
    let env = Env::default();
    env.mock_all_auths_allowing_non_root_auth();
    let key = EdSigningKey::from_bytes(&[43u8; 32]);
    let (token_addr, token, asset_admin) = create_token(&env);
    let funder = Address::generate(&env);
    let recipient = Address::generate(&env);
    asset_admin.mint(&funder, &i128::MAX);
    let channel = env.register(
        channel_contract::WASM,
        (&token_addr, &funder, &BytesN::from_array(&env,
&key.verifying_key().to_bytes()), &recipient, i128::MAX, 100u32),
    );
    let client = channel_contract::Client::new(&env, &channel);
    assert_eq!(client.deposited(), i128::MAX);
    client.settle(&i128::MAX, &commitment_sig(&env, &key, &channel, i128::MAX));
    assert_eq!(client.withdrawn(), i128::MAX);
    assert_eq!(client.deposited(), i128::MAX);

    // Recycle an existing unit: no balance or supply exceeds i128::MAX.
    token.transfer(&recipient, &funder, &1);
    client.top_up(&1);
    assert_eq!(client.balance(), 1);
    assert!(client.try_deposited().is_err());
    assert_eq!(client.withdrawn(), i128::MAX);
```

```
    client.close_start();
    env.ledger().with_mut(|l| l.sequence_number += 100);
    client.refund();
    assert_eq!(token.balance(&funder), 1);
    assert_eq!(client.balance(), 0);
    assert_eq!(client.deposited(), i128::MAX);
}
```

Manual check: rg -n "pub fn deposited|overflow-checks" contracts/channel/src/lib.rs Cargo.toml.

I-5. Channel factory implementation updates are unannounced and unbound to funder authorization

Commands:

```
cargo build --workspace --exclude ed25519-tool --release --target wasm32v1-none
cargo build --manifest-path audit-tests/report-findings/Cargo.toml --release --target wasm32v1-none
cargo test --manifest-path audit-tests/report-findings/Cargo.toml test_I5_AdminWasmSwapCanReuseOpenAuthorizationAndDrainDeposit -- --nocapture
```

Expected result: admin-authorized set_wasm changes future deployments without an event, and funder open authorization is not bound to an expected wasm hash. The test uses exact auth mocks for minting, set_wasm, and open.

Primary test function:

```
#[test]
#[allow(non_snake_case)]
fn test_I5_AdminWasmSwapCanReuseOpenAuthorizationAndDrainDeposit() {
    let env = Env::default();

    let admin = Address::generate(&env);
    let benign_hash =
env.deployer().upload_contract_wasm(channel_contract::WASM);
    let malicious_wasm = Bytes::from_slice(&env, include_bytes!("../target/wasm32v1-none/release/report_findings.wasm"));
    let malicious_hash = env.deployer().upload_contract_wasm(malicious_wasm);

    let factory_id = env.register(channel_factory_contract::WASM, (&admin, &benign_hash));
    let factory_client = channel_factory_contract::Client::new(&env, &factory_id);

    let (token_admin, token_addr, token, asset_admin) =
create_token_with_admin(&env);
    let funder = Address::generate(&env);
    let recipient = Address::generate(&env);
    let attacker = Address::generate(&env);
    let commitment_key = BytesN::from_array(&env, &[7u8; 32]);
    let salt = BytesN::from_array(&env, &[1u8; 32]);
    let amount = 500i128;
    let refund_waiting_period = 100u32;
```

```
    asset_admin
        .mock_auths(&[MockAuth {
            address: &token_admin,
            invoke: &MockAuthInvoke {
                contract: &token_addr,
                fn_name: "mint",
                args: (&funder, &amount).into_val(&env),
                sub_invokes: &[],
            },
        }])
        .mint(&funder, &amount);

    let deployment_salt: BytesN<32> =
env.crypto().sha256(&DeploymentSaltPreimage(funder.clone(),
salt.clone())).to_xdr(&env)).into();
    let channel_id = env.deployer().with_address(factory_id.clone(),
deployment_salt).deployed_address();

    factory_client
        .mock_auths(&[MockAuth {
            address: &admin,
            invoke: &MockAuthInvoke {
                contract: &factory_id,
                fn_name: "set_wasm",
                args: (&malicious_hash,).into_val(&env),
                sub_invokes: &[],
            },
        }])
        .set_wasm(&malicious_hash);

    let opened = factory_client
        .mock_auths(&[MockAuth {
            address: &funder,
            invoke: &MockAuthInvoke {
                contract: &factory_id,
                fn_name: "open",
                args: (&salt, &token_addr, &funder, &commitment_key, &recipient, &amount,
&refund_waiting_period).into_val(&env),
                sub_invokes: &[MockAuthInvoke {
                    contract: &channel_id,
                    fn_name: "__constructor",
                    args: (&token_addr, &funder, &commitment_key, &recipient, &amount,
&refund_waiting_period).into_val(&env),
                    sub_invokes: &[MockAuthInvoke {
                        contract: &token_addr,
                        fn_name: "transfer",
                        args: (&funder, &channel_id, &amount).into_val(&env),
                        sub_invokes: &[],
                    }],
                }],
            },
        }])
        .open(&salt, &token_addr, &funder, &commitment_key, &recipient, &amount,
&refund_waiting_period);
```

```
    assert_eq!(opened, channel_id);
    assert_eq!(token.balance(&channel_id), amount);
    assert_eq!(token.balance(&funder), 0);

    MaliciousChannelClient::new(&env, &channel_id).steal(&attacker);

    assert_eq!(token.balance(&channel_id), 0);
    assert_eq!(token.balance(&attacker), amount);
}
```

Manual checks:

```
rg -n "set_wasm|pub fn open|require_auth|WasmHash" contracts/channel-factory/src/lib.rs
```

Expected result:

- set_wasm emits no event.
- open deploys whatever hash is stored at execution time.

HackenProof

VULNERABILITY DETAILS

ROZOSCA-11 Informational 2

• NONE

Target https://github.com/mpprouter/one-way-channel

Status: Resolved

Severity: • NONE

Vulnerability details

[I-6] refund drains balance but does not extinguish surviving commitments

Risk Category: Channel lifecycle / stale commitments

Context:
- contracts/channel/src/lib.rs::Contract::settle() lines 456-490
- contracts/channel/src/lib.rs::Contract::refund() lines 597-615
- contracts/channel/src/lib.rs::Contract::withdraw() lines 628-645

Description: After refund drains the channel, the contract does not mark outstanding commitments as expired or consumed. If the channel later receives tokens, a previously valid commitment can still be used to withdraw up to its remaining unpaid amount.

This behavior follows from the documented design: shortfalls remain claimable after future top-ups. It becomes risky when a channel address is reused after refund or accidentally receives tokens. Unexpected post-refund balance can be paid to the recipient under an old commitment, but the recipient still cannot exceed the cumulative signed commitment amount.

Recommendation:

If refunded channels should be final, set a terminal state on refund and reject later settle, close, and top_up. If post-refund reuse is intentional, add a session / epoch to commitments so old commitments cannot apply to later channel balances.

[I-7] refund_waiting_period has no upper bound

Risk Category: Input validation / liveness

Context:
- contracts/channel/src/lib.rs::Contract::__constructor() lines 304-312
- contracts/channel/src/lib.rs::Contract::close_start() lines 577-580
- contracts/channel/src/lib.rs::Contract::refund() lines 597-601

Description: refund_waiting_period is immutable and can be set as high as u32::MAX. close_start computes the effective ledger using saturating_add, so a very large period can set the close-effective ledger near the maximum representable sequence number.

The funder chooses and authorizes this value, and the recipient is expected to verify it before accepting the channel. The missing bound is still an integration hazard because it can create channels with unusable refund timing. Funds can be locked for an impractically long time if the funder or integration chooses a bad value, although the condition is visible to the recipient and self-inflicted by the funder.

Recommendation:

Set explicit minimum and maximum supported waiting periods in ledgers or wall-clock duration. Reject values outside that range in the constructor and document the accepted range.

[I-8] Ledger-denominated durations depend on a 5-second ledger cadence

Risk Category: Blockchain integration / time assumptions

Context:

- contracts/account/src/lib.rs TTL constants, lines 56-61
- contracts/channel/src/lib.rs TTL constants, lines 215-220
- contracts/channel/src/lib.rs::Contract::__constructor() refund_waiting_period docs, lines 290-298

Description: Both contracts define LEDGERS_PER_DAY = 17280 with the comment “assuming 5 second ledger close times.” The channel also exposes refund_waiting_period directly as a ledger count.

Ledger count is the native Soroban timing primitive, but user-facing documentation describes the waiting period as time for the recipient to observe a close and react. If ledger cadence changes, the wall-clock meaning of existing ledger-denominated values changes as well. The contract correctly uses ledger sequence numbers; the practical risk is client misconfiguration or stale wall-clock assumptions.

Recommendation:

Document timing in both ledgers and approximate wall-clock time. For client APIs, accept wall-clock durations and convert them using current network assumptions at channel creation, while storing and displaying the final ledger count. For long-lived channels, warn that immutable ledger counts do not automatically adjust if network cadence changes.

[I-9] Event fields needed for discovery and monitoring are not indexed as topics

Risk Category: Event indexing / monitoring

Context:

- contracts/channel/src/event.rs::Open lines 3-19
- contracts/channel/src/event.rs::Deposit lines 21-34
- contracts/channel/src/event.rs::Close lines 36-47
- contracts/channel/src/event.rs::Withdraw / Refund lines 49-68

Description: The event structs do not mark any field with #[topic]. Soroban events can be filtered by contract ID and topics. With only the event-name topic available, a recipient cannot query directly for all events where to is their address unless they already know every channel contract to watch.

This matters because the module-level expectations place monitoring responsibility on the recipient, and payment-processor usage naturally involves many payer-created channel addresses. Recipient-side discovery and monitoring are therefore more dependent on out-of-band channel registration. Funds are not directly at risk when a recipient already tracks the channel address.

Recommendation:

Index the fields that clients need for discovery and monitoring. Recommended topics include to for Open and Withdraw, from for Open, Deposit, and Refund, and effective_at_ledger only if ledger-range filtering is useful to the intended indexer model.

Example:

```
pub struct Open {
    #[topic]
    pub from: Address,
    #[topic]
    pub to: Address,
    ...
}
```

Validation steps

I-6. refund drains balance but does not extinguish surviving commitments

Commands:

```
cargo test --manifest-path audit-tests/report-findings/Cargo.toml
test_I6_PostRefundDepositRemainsClaimable -- --nocapture
```

Expected result:

- After refund, withdrawn() remains below the old commitment amount.
- New balance arriving at the same channel can be claimed by the recipient using the old commitment.

Test function:

```
#[test]
#[allow(non_snake_case)]
fn test_I6_PostRefundDepositRemainsClaimable() {
    let env = Env::default();
    env.mock_all_auths_allowing_non_root_auth();

    let commitment_key = EdSigningKey::from_bytes(&[3u8; 32]);
    let commitment_pub = BytesN::from_array(&env,
&commitment_key.verifying_key().to_bytes());

    let (token_addr, token, asset_admin) = create_token(&env);
    let funder = Address::generate(&env);
    let to = Address::generate(&env);
    asset_admin.mint(&funder, &1_300);

    let channel = env.register(channel_contract::WASM, (&token_addr, &funder,
&commitment_pub, &to, 1_000i128, 100u32));
    let client = channel_contract::Client::new(&env, &channel);

    // The funder has signed a commitment for the full 1000. The recipient
    // never presents it.
    let sig = commitment_sig(&env, &commitment_key, &channel, 1_000i128);

    // Funder closes out and reclaims everything after the waiting period.
    client.close_start();
    env.ledger().with_mut(|l| l.sequence_number += 101);
    client.refund();
```

```
    assert_eq!(token.balance(&channel), 0);
    assert_eq!(token.balance(&funder), 1_300);
    assert_eq!(client.withdrawn(), 0); // <-- the stale claim is still worth 1000

    // Any later balance at this address - a mistaken re-funding by the funder,
    // an automated payer that still points at the old channel, a third party -
    // is immediately claimable by the recipient with the old commitment.
    token.transfer(&funder, &channel, &300i128);
    client.settle(&1_000i128, &sig);

    assert_eq!(token.balance(&to), 300);
    assert_eq!(token.balance(&channel), 0);
    assert_eq!(client.withdrawn(), 300);
    std::println!("recipient claimed {} from a channel the funder had already refunded",
token.balance(&to));
}
```

Manual checks:

```
rg -n "pub fn refund|fn withdraw|WithdrawnAmount|settle" contracts/channel/src/
lib.rs
```

Expected result:

- refund transfers balance but does not update WithdrawnAmount or invalidate commitments.
- withdraw keeps shortfalls claimable by advancing only actual payout.

I-7. refund_waiting_period has no upper bound

Commands:

```
cargo test --manifest-path audit-tests/report-findings/Cargo.toml
test_I7a_UnboundedWaitingPeriodDefersRefundToLedgerCeiling -- --nocapture
cargo test --manifest-path audit-tests/report-findings/Cargo.toml
test_I7b_SaturationPlusBadKeyLocksEscrowForBothSides -- --nocapture
cargo test --manifest-path audit-tests/report-findings/Cargo.toml
test_I7c_ZeroWaitingPeriodGivesRecipientNoWindow -- --nocapture
cargo test --manifest-path audit-tests/report-findings/Cargo.toml
test_I7d_OversizedWaitingPeriodKeepsRefundUnreachableAfterCenturyAdvance --
--nocapture
```

Expected result:

- A very large waiting period pushes refund reachability to an impractical ledger height.
- Combining an oversized period with an unredeemable commitment key blocks practical exit for both sides.
- A zero period gives the recipient no reaction window.
- test_I7d_OversizedWaitingPeriodKeepsRefundUnreachableAfterCenturyAdvance pa sses and prints the oversized-period liveness counterexample.

Primary test function:

```
#[test]
#[allow(non_snake_case)]
fn test_I7b_SaturationPlusBadKeyLocksEscrowForBothSides() {
    let env = Env::default();
    env.mock_all_auths_allowing_non_root_auth();
    env.ledger().with_mut(|l| l.sequence_number = 55_000_000);
```

```
    let mut identity = [0u8; 32];
    identity[0] = 1;
    let commitment_key = BytesN::from_array(&env, &identity);
    let mut forged = [0u8; 64];
    forged[0] = 1;
    let sig = BytesN::from_array(&env, &forged);

    let (token_addr, token, asset_admin) = create_token(&env);
    let funder = Address::generate(&env);
    let to = Address::generate(&env);
    asset_admin.mint(&funder, &1_000);

    let period: u32 = 4_290_000_000;
    let channel = env.register(channel_contract::WASM, (&token_addr, &funder,
&commitment_key, &to, 1_000i128, period));
    let client = channel_contract::Client::new(&env, &channel);

    client.close_start();
    env.ledger().with_mut(|l| l.sequence_number += 630_720_000);

    assert!(client.try_refund().is_err(), "saturated close keeps the funder's refund
unreachable");
    assert!(client.try_close(&0i128, &sig).is_err(), "small-order key blocks recipient
close");
    assert!(client.try_settle(&1_000i128, &sig).is_err(), "small-order key blocks recipient
settle");
    assert_eq!(token.balance(&channel), 1_000);
}
```

Manual checks:

```
rg -n "refund_waiting_period|saturating_add|RefundWaitingPeriod" contracts/
channel/src/lib.rs
```

Expected result:

- The constructor stores the period without min/max bounds.
- close_start computes the effective ledger with saturating_add.

I-8. Ledger-denominated durations depend on a 5-second ledger cadence

Validation type: inspection-only. No standalone audit-package test is attached because this finding concerns ledger-unit assumptions and client configuration.

Commands:

```
rg -n "LEDGERS_PER_DAY|5 second|refund_waiting_period|sequence" contracts/
account/src/lib.rs contracts/channel/src/lib.rs
```

Expected result:

- Account and channel TTL constants assume 5-second ledgers.
- refund_waiting_period is a raw ledger count and is immutable for a channel.
- No timestamp or dynamic cadence adjustment is used.

External checks used:

- Stellar Protocol 23 upgrade guide.
- CAP-0070.
- SDF roadmap material for lower-latency ledger targets.

Expected conclusion:

- The contract uses the correct native ledger primitive, but clients must not present ledger counts as stable wall-clock guarantees without current-network conversion and documentation.

I-9. Event fields needed for discovery and monitoring are not indexed as topics

Validation type: inspection-led. No standalone audit-package test is attached;
the production event test is used only to confirm the current event layout.

Commands:

```
rg -n "#\\[topic\\]|contractevent|pub struct Open|pub struct Close|pub struct
Withdraw|pub struct Refund" contracts/channel/src/event.rs
cargo test -p channel test_top_up_emits_deposit_event
```

Expected result:

- No event field in event.rs is marked with #[topic].
- Channel event snapshots show the event name as the topic while address fields are stored in data.

Sanity test function:

```
#[test]
fn test_top_up_emits_deposit_event() {
    let env = Env::default();
    env.mock_all_auths();

    let auth_key = SigningKey::from_bytes(&[29u8; 32]);
    let auth_pubkey = BytesN::from_array(&env, &auth_key.verifying_key().to_bytes());

    let to = Address::generate(&env);
    let funder = Address::generate(&env);

    let (token_addr, _token, asset_admin) = create_token(&env);
    asset_admin.mint(&funder, &1000);

    let channel_id = env.register(Contract, (token_addr.clone(), funder.clone(),
auth_pubkey.clone(), to.clone(), 500i128, 100u32));
    let client = ContractClient::new(&env, &channel_id);

    client.top_up(&200);
    assert!(has_event_type(&env, &channel_id, "deposit"));
}
```

Manual checks:

```
rg -n "Monitors the channel|Close events" contracts/channel/src/lib.rs
```

Expected result:

- Recipient monitoring is a documented responsibility, but event filtering by recipient address is not supported by the current event topic layout.

HackenProof

VULNERABILITY DETAILS

ROZOSCA-8 Opaque `personal_sign` authorizations hide the action being approved

• NONE

Target https://github.com/mpprouter/one-way-channel

Status: Resolved

Severity: • NONE

Vulnerability details

[L-1] Opaque personal_sign authorizations hide the action being approved

Risk Category: Signature authorization/user-consent integrity

Context:

- contracts/account/src/lib.rs signed-message format, lines 21-37
- contracts/account/src/lib.rs::Contract::__check_auth() lines 128-168

Description: The account contract verifies an EIP-191 personal_sign signature over a 32-byte Soroban authorization payload rendered as a hex string. Soroban binds the payload to the invocation tree, nonce, network, and expiration, so this is not a raw replay issue. The weakness is consent visibility: a browser wallet prompt shows an opaque hex payload, not the action being authorized.

The contract also receives _auth_contexts, but intentionally does not inspect or restrict them:

```
fn __check_auth(env: Env, signature_payload: Hash<32>, signature: BytesN<65>,
_auth_contexts: Vec<Context>) -> Result<(), Error> {
    ...
}
```

A malicious frontend can ask a user to sign a payload while describing it as a harmless action. If the signature corresponds to a token transfer, channel opening, close_start, or refund, the account will authorize that invocation.

Impact: A user can be tricked into authorizing an action they did not understand from the wallet prompt. The attacker still needs the exact user signature for the exact Soroban payload, so this is a phishing/user-consent issue rather than an autonomous contract exploit.

Proof of Concept: Simplified scenario:

1. A user controls a Soroban account through an EVM wallet.
2. A frontend asks the wallet to personal_sign an opaque 66-character hex string.
3. The signed payload authorizes a sensitive Soroban invocation.
4. __check_auth accepts the signature because the EVM address matches.

Retained validation:

- test_L1a_ChosenMessageSignatureAuthorizesTransfer
- test_L1b_ChosenMessageSignatureAuthorizesChannelAction
- test_L1c_WrongKeyIsRejected

Recommendation:

Do not rely on raw personal_sign prompts as the only consent surface for sensitive account actions. At minimum, first-party clients should decode and display the Soroban authorization contexts before requesting the signature. For stronger protection, use a structured signing flow or wallet integration that presents the action, asset, amount, destination, contract, network, and expiration in a human-readable form before signing.

Validation steps

L-1. Opaque personal_sign authorizations hide the action being approved

Commands:

```
cargo test --manifest-path audit-tests/report-findings/Cargo.toml
test_L1a_ChosenMessageSignatureAuthorizesTransfer -- --nocapture
cargo test --manifest-path audit-tests/report-findings/Cargo.toml
test_L1b_ChosenMessageSignatureAuthorizesChannelAction -- --nocapture
cargo test --manifest-path audit-tests/report-findings/Cargo.toml
test_L1c_WrongKeyIsRejected -- --nocapture
cargo test -p account test_check_auth_high_s
cargo test -p account test_check_auth_wrong_signer
```

Expected result:

- A user-signed opaque hex payload authorizes a token transfer.
- The same payload style authorizes a channel action.
- Wrong signer and high-s signatures are rejected, confirming the issue is consent visibility rather than broken signature recovery.

Primary test function:

```
#[test]
#[allow(non_snake_case)]
fn test_L1a_ChosenMessageSignatureAuthorizesTransfer() {
    let env = Env::default();
    env.mock_all_auths_allowing_non_root_auth();

    // --- Victim setup: an EVM key controlling a Soroban account holding funds.
    let victim_key = SigningKey::from_bytes((&[11u8; 32]).into()).unwrap();
    let eth = BytesN::from_array(&env, &eth_address(&victim_key));

    let account_wasm =
env.deployer().upload_contract_wasm(account_contract::WASM);
    let factory = env.register(account_factory_contract::WASM, (&account_wasm,));
    let account = account_factory_contract::Client::new(&env,
&factory).open_account(&eth);

    let (token_addr, token, asset_admin) = create_token(&env);
    asset_admin.mint(&account, &1_000);
    assert_eq!(token.balance(&account), 1_000);
```

```
    // --- Attacker: pick the invocation to steal with, and derive the exact
    //     66-character string the victim must be induced to sign.
    let attacker = Address::generate(&env);
    let nonce = 777i64;
    // The attacker picks the expiry, up to the network max entry TTL, so a
    // phished signature can be banked and used months later.
    let exp = env.ledger().sequence() + 3_110_000;
    let theft = contract_fn(&token_addr, "transfer", std::vec![sc_val(&env, &account),
sc_val(&env, &attacker), sc_val(&env, 1_000i128)]);
    let payload = soroban_auth_payload(&env, nonce, exp, &theft);
    let challenge = hex_0x(&payload);

    // The attacker's site presents this as an ordinary sign-in nonce.
    assert_eq!(challenge.len(), 66);
    std::println!("phishing site shows:  \"Sign in\"  message = {challenge}");

    // --- Victim: signs the challenge with a stock wallet. No Soroban involved.
    let signature = personal_sign(&victim_key, &challenge);

    // --- Attacker: replays that signature as a Soroban authorization.
    env.set_auths(&[entry(&account, nonce, exp, signature, theft)]);
    token.transfer(&account, &attacker, &1_000i128);

    assert_eq!(token.balance(&account), 0);
    assert_eq!(token.balance(&attacker), 1_000);
    std::println!("account drained: attacker balance = {}", token.balance(&attacker));
}
```

Manual checks:

```
rg -n "__check_auth|personal_sign|_auth_contexts|Ethereum Signed" contracts/
account/src/lib.rs
```

Expected result:

- __check_auth reconstructs an EIP-191 message from the Soroban payload and does not inspect _auth_contexts.

ABOUT PROJECT

MPP Router is a payment-routing layer for merchants and AI-agent services on Stellar. The audited components are its Soroban smart contracts (Rust): a one-way payment channel, in which a funder deposits once and then authorizes many off-chain payments by signing commitments for increasing cumulative totals, with unilateral exit via close_start / refund.

DISCLAIMER

HackenProof Disclaimer

The application provided for assessment has been analyzed in accordance with generally accepted industry practices applicable at the time this report was prepared. The assessment focused on identifying cybersecurity vulnerabilities and security issues in the application's

source code, deployment, configuration, and functionality, based solely on the materials, access, and scope made available for the engagement.

This report does not provide any representation or warranty that all vulnerabilities, weaknesses, or misconfigurations have been identified, nor does it guarantee the absolute security, reliability, or bug-free status of the application. The conclusions herein apply only to the specific version and scope reviewed and may no longer remain valid after any modification, update, or environmental change. This report should not be regarded as a definitive or exhaustive statement of the application's security posture.

Technical Disclaimer

Applications, whether decentralized applications (DApps) or other software systems, are deployed and operated within technical environments that may include blockchains, cloud services, operating systems, programming languages, frameworks, libraries, APIs, third-party services, and infrastructure components. Any of these elements may contain inherent weaknesses, undisclosed vulnerabilities, or configuration issues that can result in security incidents, operational failures, or unauthorized access.

The assessment reflects the condition of the application only at the time of testing and within the agreed scope, and the absence of findings in this report must not be interpreted as evidence that no vulnerabilities exist. Continuous security practices, including secure development, independent reviews, monitoring, patch management, and periodic reassessments, remain necessary to maintain an appropriate level of security.

HackenProof

Expert crowdsourced security provider

PRIVATE SECURITY AUDIT

Contact us:

Linkedin | X | Website | Mail

2026